Chapter 8.5 & 9.1

Adwords Implementation
A Model for Recommendation Systems

248251 Chaewon Kim
chaewonk215@gamil.com

AGENDA

Chapter 8.5

Adwords Implementation

1. Matching Bids and Search Queries
2. More Complex Matching Problems
3. A Matching Algorithm for Documents and Bids

8.5.1

Matching Bids and Search Queries

Steps for matching bids and search queries

1. Advertisers bid on sets of words
2. Check that the search query includes words sets
3. If query contain words sets, this bid becomes a candidate for selection

Steps for matching bids and search queries

1. **Advertisers bid on sets of words**
2. Check that the search query includes words sets
3. If query contain words sets, this bid becomes a candidate for selection

Advertiser	Ad text		Sets of words
Nike Store	“Official Nike Store - Free Shipping”	⟶	{“nike”, “running”, “shoes”}
FootLocker	“Nike Running Shoes 20% Off!”	⟶	{“nike”, “running”, “shoes”}
SportsDirect	“Great Deals on Nike Shoes”	⟶	{“nike”, “shoes”}

Steps for matching bids and search queries

1. **Advertisers bid on sets of words**
2. Check that the search query includes words sets
3. If query contain words sets, this bid becomes a candidate for selection

Hash keys (set of words)		Hash buckets (bids)	
{“nike”, “running”, “shoes”}	⟶	Nike Store	“Official Nike Store - Free Shipping”
		FootLocker	“Nike Running Shoes 20% Off!”
{“nike”, “shoes”}	⟶	SportsDirect	“Great Deals on Nike Shoes”

Steps for matching bids and search queries

1. Advertisers bid on sets of words
2. **Check that the search query includes words sets**
3. If query contain words sets, this bid becomes a candidate for selection

Query

running shoes nike

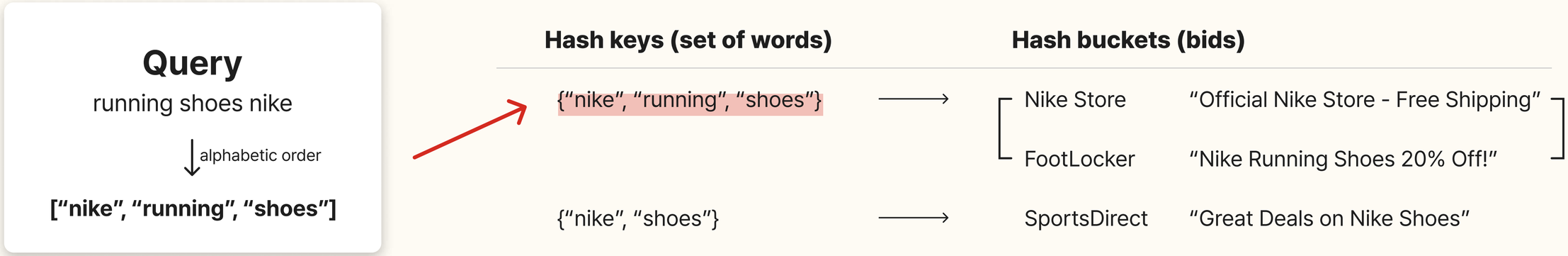
↓ alphabetic order

[“nike”, “running”, “shoes”]

Hash keys (set of words)		Hash buckets (bids)	
{“nike”, “running”, “shoes”}	⟶	Nike Store	“Official Nike Store - Free Shipping”
		FootLocker	“Nike Running Shoes 20% Off!”
{“nike”, “shoes”}	⟶	SportsDirect	“Great Deals on Nike Shoes”

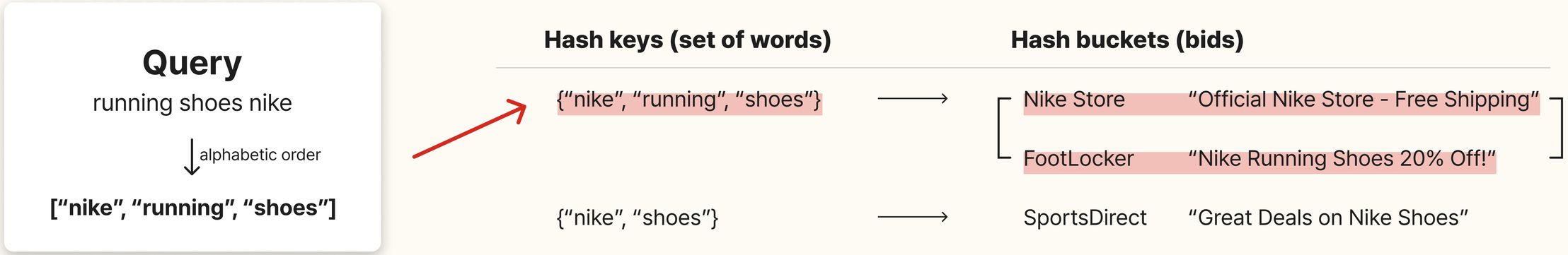
Steps for matching bids and search queries

1. Advertisers bid on sets of words
2. **Check that the search query includes words sets**
3. If query contain words sets, this bid becomes a candidate for selection



Steps for matching bids and search queries

1. Advertisers bid on sets of words
2. Check that the search query includes words sets
3. **If query contain words sets, this bid becomes a candidate for selection**



Memory used for matching and parallel process

A million advertisers * 100 bids * 100 bytes

1,000,000 * 100 * 100 = 10^{10}Bytes = **10GB**

→ Size that can be handled by a single machine's memory

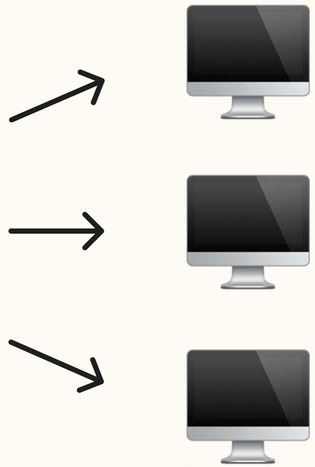
Hash keys (set of words)		Hash buckets (bids)	
{“nike”, “running”, “shoes”}	⟶	Nike Store	“Official Nike Store - Free Shipping”
⋮			⋮
{“nike”, “discount”}	⟶	FootLocker	“Nike Running Shoes 20% Off!”

Memory used for matching and parallel process

If **more main memory space** is required:

Split the buckets of the hash table among as many machines as we need

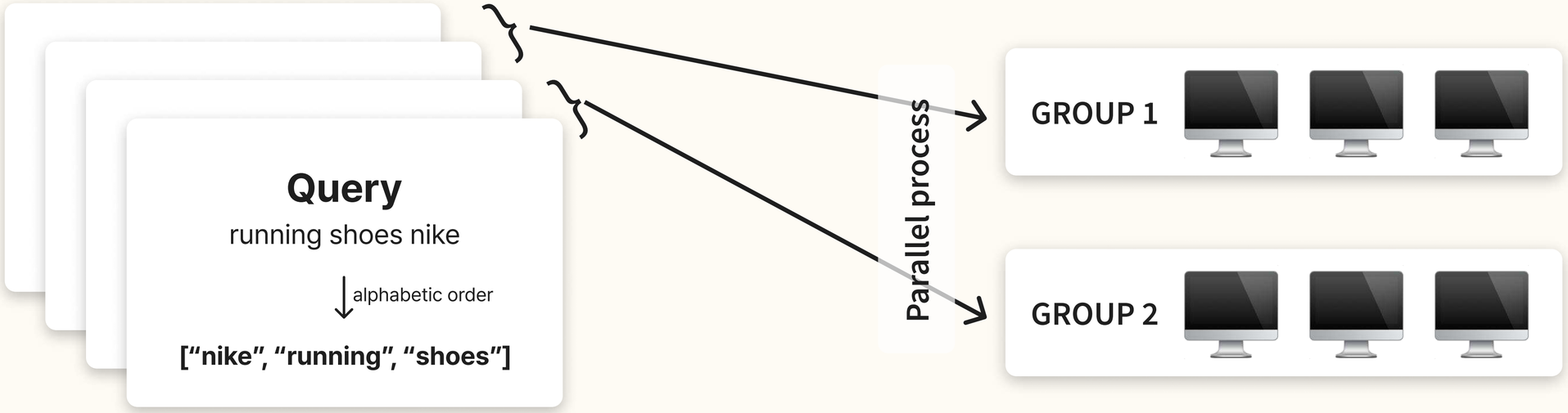
Hash keys (set of words)		Hash buckets (bids)	
{“nike”, “running”, “shoes”}	⟶	Nike Store	“Official Nike Store - Free Shipping”
⋮			⋮
{“nike”, “shoes”}	⟶	SportsDirect	“Great Deals on Nike Shoes”
⋮			⋮
{“nike”, “discount”}	⟶	FootLocker	“Nike Running Shoes 20% Off!”



Memory used for matching and parallel process

If **search queries** are arriving **too rapidly** to process them all:

- Split the stream of queries as many pieces as necessary
- Each piece is handled by one group of machines
- Parallel processing is needed to handle entire queries in main memory



8.5.2

More Complex Matching Problems

Matching bids and emails

Hi, I'm planning a trip to LA with my family next weekend.
It's a 4-day, 3-night trip and we need a rental car and are looking for a hotel.

Bid keywords

Advertiser A	{“LA”, “hotel”}
Advertiser B	{“rental car”, “LA”}
Advertiser C	{“LA”, “package”}

Matching bids and emails

Hi, I'm planning a trip to LA with my family next weekend.
It's a 4-day, 3-night trip and we need a rental car and are looking for a hotel.

Bid keywords

Advertiser A	{“LA”, “hotel”}	⟶	Match
Advertiser B	{“rental car”, “LA”}	⟶	Match
Advertiser C	{“LA”, “package”}	⟶	Mismatch

Matching bids and emails

Hi, I'm planning a trip to LA with my family next weekend.
It's a 4-day, 3-night trip and we need a rental car and are looking for a hotel.

Bid keywords

Advertiser A	{"LA", "hotel"}	⟶	Match
Advertiser B	{"rental car", "LA"}	⟶	Match
Advertiser C	{"LA", "package"}	⟶	Mismatch

As emails get **longer**, checking for matches becomes very **complex**

The reason that matching bids and emails is difficult

Need to generate and compare all subsets of email words to check for matching

Hi, I'm planning a trip to LA with my family next weekend.
It's a 4-day, 3-night trip and we need a rental car and are looking for a hotel.

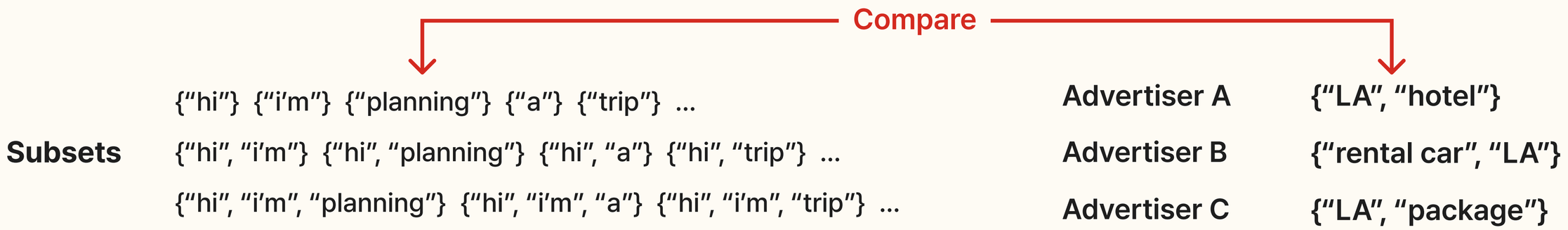
Subsets

{“hi”} {“i’m”} {“planning”} {“a”} {“trip”} ...
{“hi”, “i’m”} {“hi”, “planning”} {“hi”, “a”} {“hi”, “trip”} ...
{“hi”, “i’m”, “planning”} {“hi”, “i’m”, “a”} {“hi”, “i’m”, “trip”} ...

$\longrightarrow 2^n$

The reason that matching bids and emails is difficult

Need to generate and compare all subsets of email words to check for matching



The reason that matching bids and emails is difficult

Need to generate and compare all subsets of email words to check for matching

Subsets

{“hi”} {“i’m”} {“planning”} {“a”} {“trip”} ...

{“hi”, “i’m”} {“hi”, “planning”} {“hi”, “a”} {“hi”, “trip”} ...

{“hi”, “i’m”, “planning”} {“hi”, “i’m”, “a”} {“hi”, “i’m”, “trip”} ...

Hashing

Hash table

Hash keys (set of words)		Hash buckets (bids)	
{“LA”, “hotel”}	⟶	Advertiser A	“LA Hotel for Family Holidays”
{“rental car”, “LA”}	⟶	Advertiser B	“LA Rental car 10% Off”
{“LA”, “package”}	⟶	Advertiser C	“LA Vacation Package”

Exceeding the limits of memory and computational resources

Standing Queries

Queries that users post to a site, expecting the site to notify them whenever **something matching the query becomes available at the site**

Standing Queries

Twitter: follow Tweets that contain a set of words you specify

a set of words

{“iPod”, “free”, “music”}

Tweets

How can i get iPod music for free?

Is there any free music available on the iPod?

...

Standing Queries

Online news sites: alert for news containing specific keywords

This is simpler than email-bid matching

1. News alerts only match single words or consecutive words
2. More limited range of words than in email

keywords

“Climate change”

News articles

The global summit on climate change addresses urgent issues → 🔔

Experts discuss climate effects and the need for systemic change → 🔕

8.5.3

A Matching Algorithm for Documents and Bids

Matching many bids and many documents

Bids

a small set of words

- hundred million or a billion of bids
- processing in main memory

Documents

a larger set of words

- hundreds of documents per second arriving
- split documents into streams for handling

Matching many bids and many documents

Step 1. Sorting words in document

Welcome to the new Apple Store and check our latest iPhone Pro deals

A list of the 100,000 most common words on the web

- to, the, and, is, in, of, for, with, on, at, ..., new, ...

Matching many bids and many documents

Step 1. Sorting words in document

Welcome to the new Apple Store and check our latest iPhone Pro deals

A list of the 100,000 most common words on the web

- to, the, and, is, in, of, for, with, on, at, ..., new, ...

new	and	the	to
Least freq.			Most freq.

Matching many bids and many documents

Step 1. Sorting words in document

Welcome to the new Apple Store and check our latest iPhone Pro deals

Apple check deals iPhone latest our Pro Store Welcome new and the to

Alphabetic order — Least freq. — Most freq.

Matching many bids and many documents

Step 1. Sorting words in document

Welcome to the new Apple Store and check our latest iPhone Pro deals

↓

Apple check deals iPhone latest our Pro Store Welcome new and the to

Matching many bids and many documents

Step 2. Hashing and matching words

Bids

- Advertiser A: "iPhone Pro Max"
- Advertiser B: "Apple deals"
- Advertiser C: "new iPhone"

Matching many bids and many documents

Step 2. Hashing and matching words

Using two hash table

Hash Table A

- save all bids
- status = 0

Hash Table B

- save bids that have been partially matched
- status ≥ 1

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- (Empty)

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- (Empty)

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- (Empty)

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- "deals" → {"Apple deals", status: 1}

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- "deals" → {"Apple deals", status: 1}

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- "deals" → {"Apple deals", status: 1}

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- "deals" → {"Apple deals", status: 1}

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- "deals" → {"Apple deals", status: 1}

↓

📁 Go to the list of completed matches

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- (Empty)

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- (Empty)

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- "Pro" → {"iPhone Pro Max", status: 1}

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- "Pro" → {"iPhone Pro Max", status: 1}

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- "Pro" → {"iPhone Pro Max", status: 1}

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- "Max" → {"iPhone Pro Max", status: 2}

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- "Max" → {"iPhone Pro Max", status: 2}

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- "Max" → {"iPhone Pro Max", status: 2}

Matching many bids and many documents

Step 2. Hashing and matching words

Apple check deals iPhone latest our Pro Store Welcome new and the to

Hash Table A

- "iPhone" → {“iPhone Pro Max”, status: 0}
- "Apple" → {“Apple deals”, status: 0}
- "new" → {“new iPhone”, status: 0}

Hash Table B

- "Max" → {"iPhone Pro Max", status: 2}
- "new" → {"new iPhone", status: 1}

Matching many bids and many documents

Step 2. Hashing and matching words

List of completed matches

Bids

- Advertiser A: "iPhone Pro Max"
- **Advertiser B: "Apple deals"**
- Advertiser C: "new iPhone"

Matching many bids and many documents

Summary

1. Each word is searched in two hash tables (B→A)
2. Partially matched bids will have their status updated and rehashed
3. Fully matched bids move to the final results list

The benefit of the rarest-first order

1. A bid is only copied to the second hash table if its rarest word appears in the document
   → Reduces main memory usage compared to alphabetical order.
2. This increases the likelihood of keeping the entire table in main memory.

AGENDA

Chapter 9.1

A Model for Recommendation Systems

1. The Utility Matrix
2. The Long Tail
3. Applications of Recommendation Systems
4. Populating the Utility Matrix

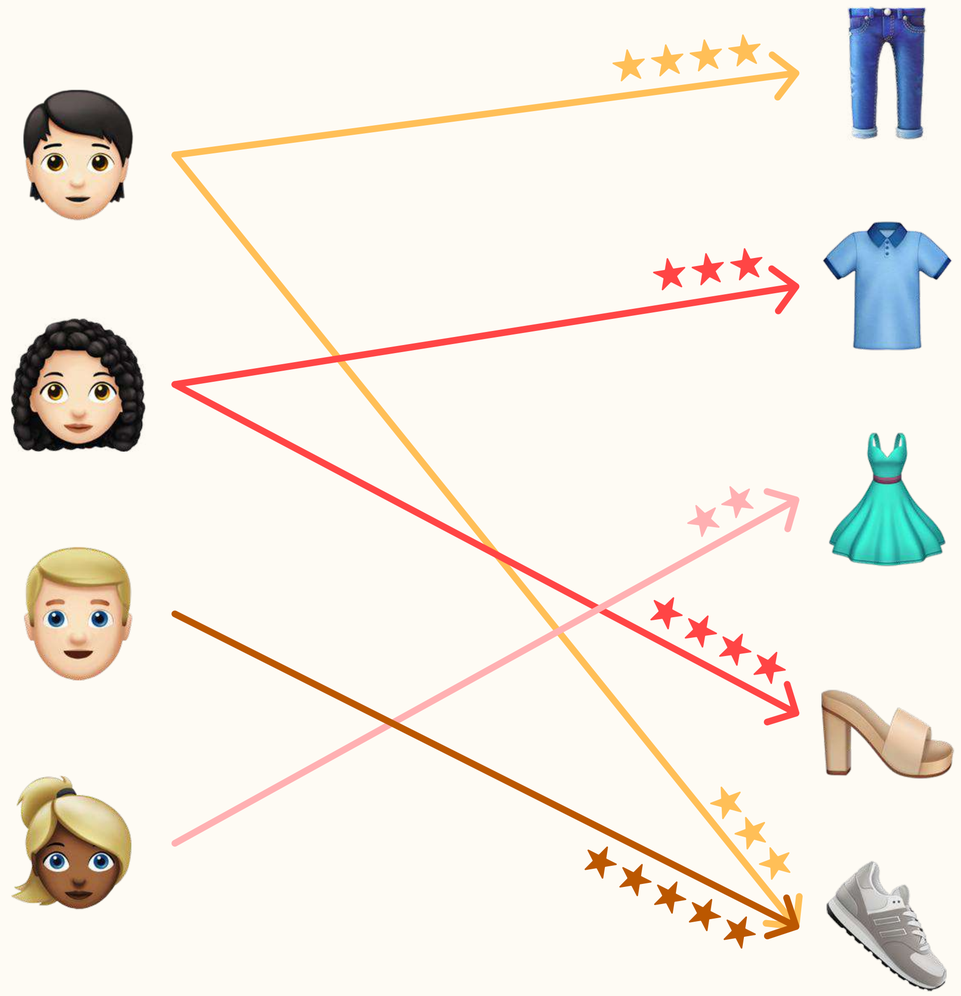
9.1.1

The Utility Matrix

Utility Matrix

Entity: Users and Items

Value: the degree of preference of that user for that item



Utility Matrix

Entity: Users and Items

Value: the degree of preference of that user for that item

					
	4				3
		3		4	
					5
			2		

Utility Matrix

Entity: Users and Items

Value: the degree of preference of that user for that item

					
	4	?	?	?	3
	?	3	?	4	?
	?	?	?	?	5
	?	?	2	?	?

Utility Matrix

Entity: Users and Items

Value: the degree of preference of that user for that item

					
	4	?			3
		3		4	?
		?			5
	?		2	?	

Utility Matrix

Entity: Users and Items

Value: the degree of preference of that user for that item

	👖	👕	👗	👡	👟
👩🏻	4	?			3
👩🏻		3		4	?
👱🏻		?			5
👱🏽‍♀️	?		2	?	

only need to predict preferences for items that are likely to have high value for each user

9.1.2

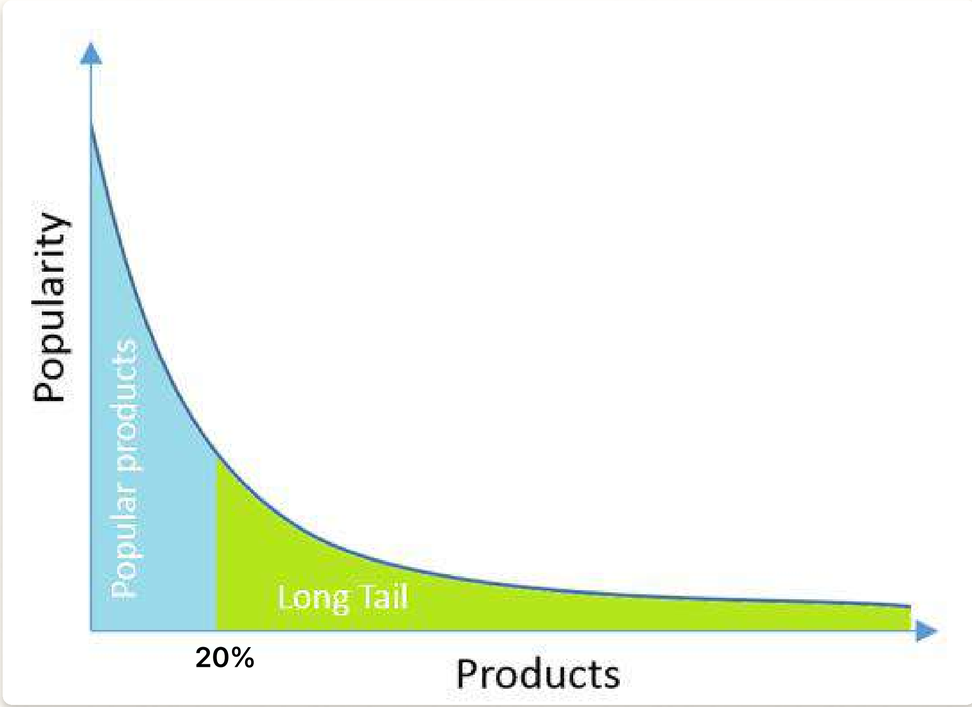
The Long Tail

The necessity for a recommendation system

The difference between offline and online = Long tail phenomenon

🏢 **Offline:** limited space
→ Focus on the **top 20%** of the most popular items

🌐 **Online:** unlimited space
→ **Recommend** items to individual users across the **entire items**



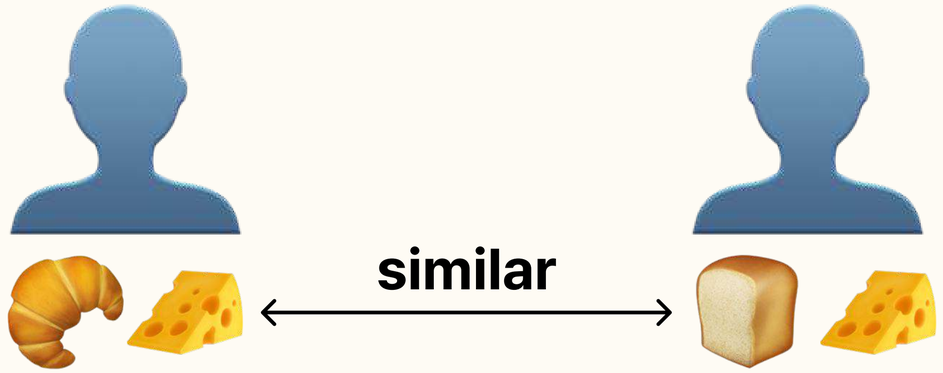
9.1.3

Applications of Recommendation Systems

1. Product Recommendations

amazon

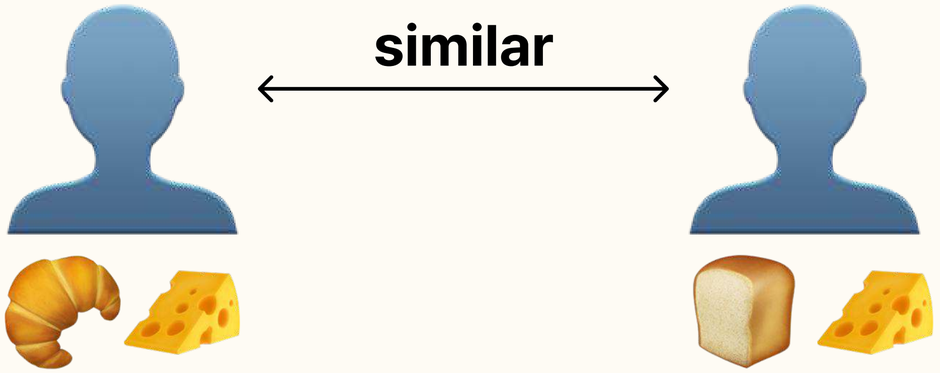
Based on the **purchasing decisions** made by **similar customers**



1. Product Recommendations

amazon

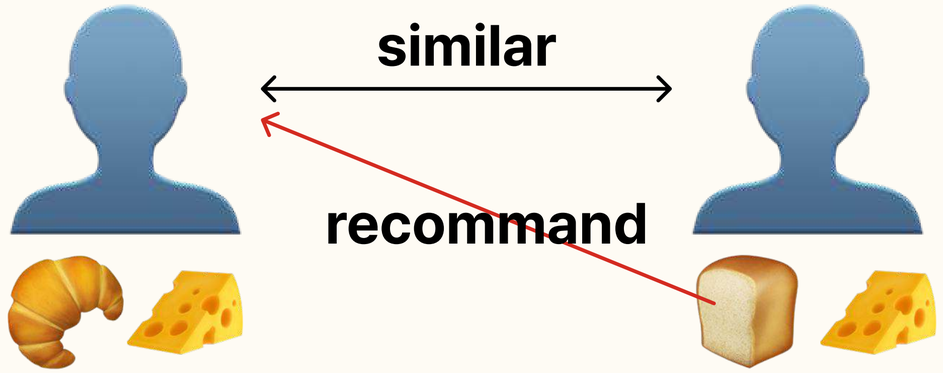
Based on the **purchasing decisions** made by **similar customers**



1. Product Recommendations

amazon

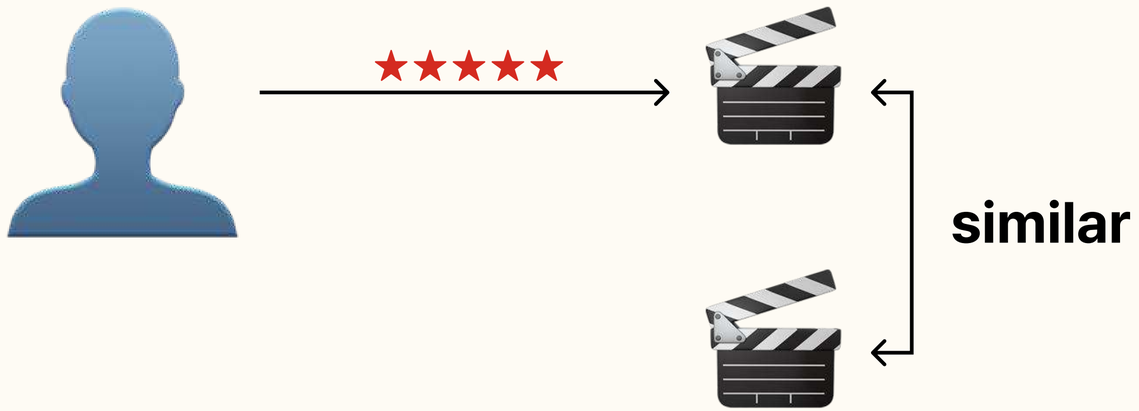
Based on the **purchasing decisions** made by **similar customers**



2. Movie Recommendations

NETFLIX

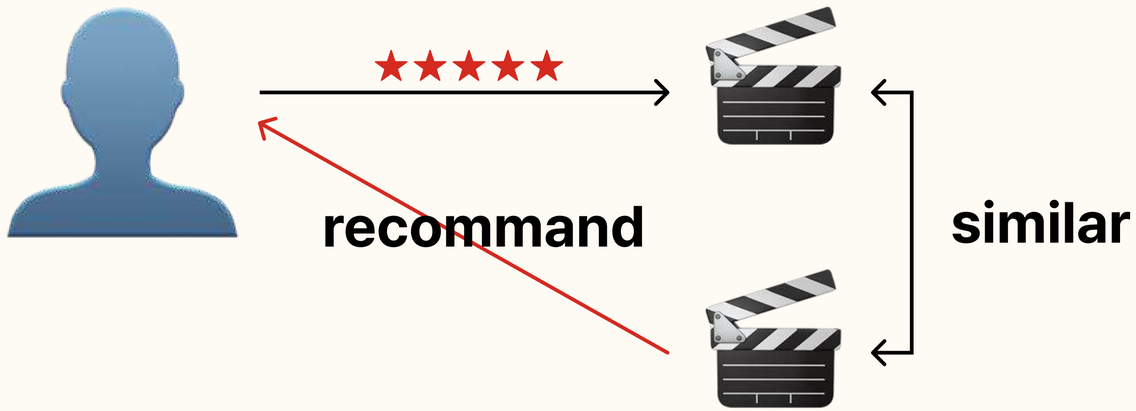
Based on **ratings** provided by **users**



2. Movie Recommendations

NETFLIX

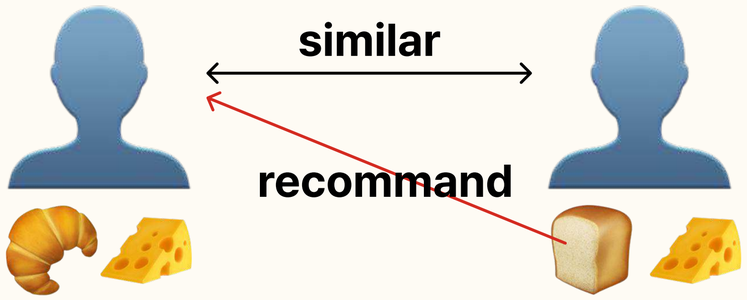
Based on **ratings** provided by **users**

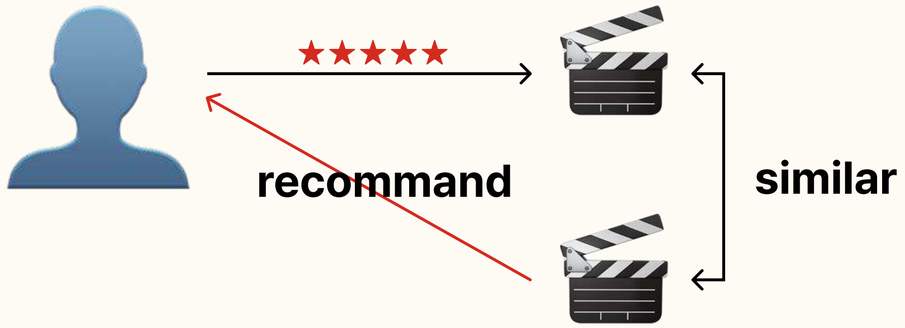


3. News Articles

Based on the **articles** that they have **read in the past**

- Similarity between articles or users





9.1.4

Populating the Utility Matrix

How user preferences are collected

1. **Asking users to rate items**
   - Users are unwilling to provide responses
   - The information may be biased

2. **Inferring from user's behavior**
   - Users can be said to "like" an item, by taking actions such as purchasing, watching, and reading
   - For example, Amazon infers users' preferences by viewing products, not just when they buy them

Thank you

248251 Chaewon Kim
chaewonk215@gamil.com